NIT2112
Object Oriented Programming

Session 02

Encapsulation and Information Hiding

VICTORIA UNIVERSITY
MELBOURNE AUSTRALIA

Acknowledgement of Country

Victoria University acknowledges, recognises and respects the Ancestors, Elders and families of the Bunurong/Boonwurrung, Wadawurrung and Wurundjeri/Woiwurrung of the Kulin who are the traditional owners of University land in Victoria, the Gadigal and Guring-gai of the Eora Nation who are the traditional owners of University land in Sydney, and the Yugara/YUgarapul people and Turrbal people living in Meanjin (Brisbane).

Session Outline

- Encapsulation: Definition & Benefits
- Access Modifiers (Public, Protected, Private)
- Information Hiding: Getters and Setters
- Name Mangling Explained
- Encapsulation Best Practices

Encapsulation: Bundling and Hiding

- **Encapsulation** is the bundling of **data (attributes)** and **methods** that operate on that data into a single unit (a class). It also involves **hiding** the internal state of an object and requiring all interactions to be performed through the object's methods.

- A capsule containing medicine – you only interact with the capsule as a whole, not the individual ingredients directly.

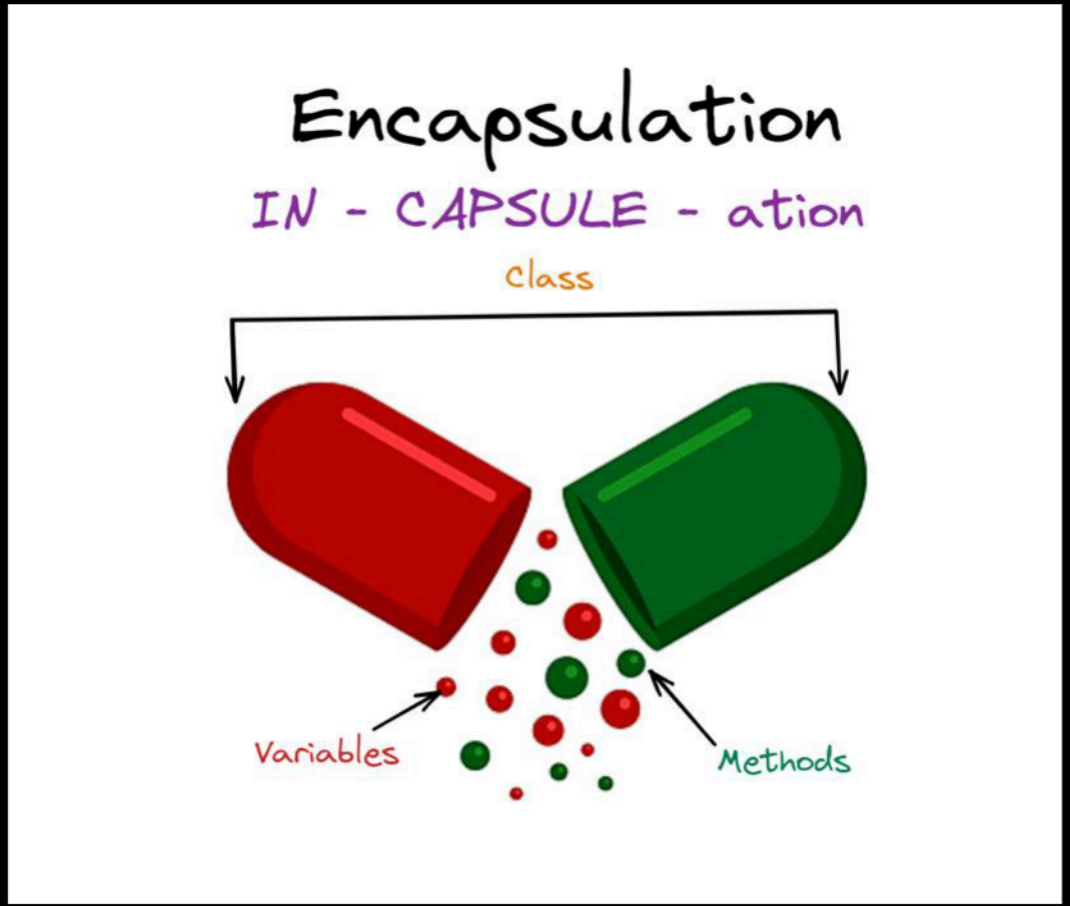


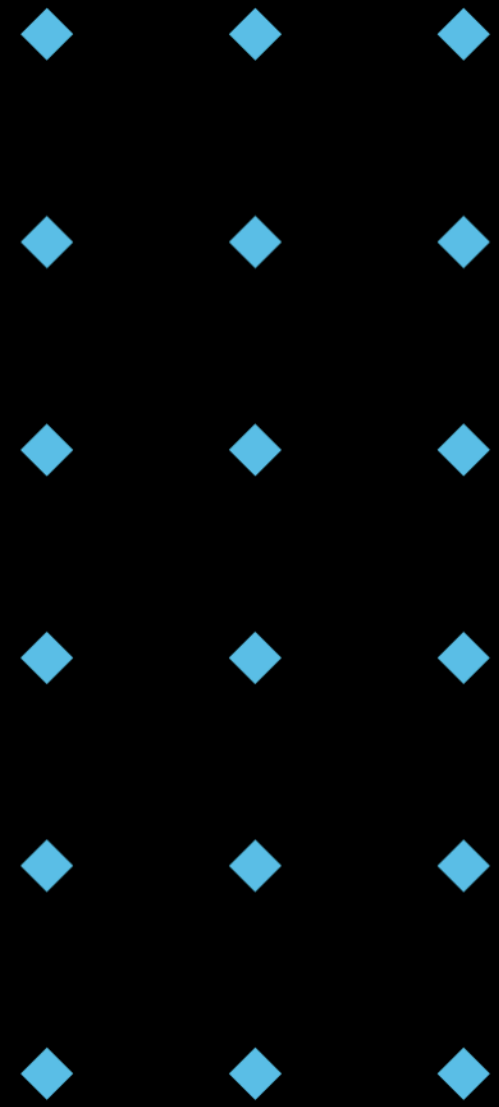
Image from AlgoDaily, year unknown,
https://algodaily.com/lessons/understanding-encapsulation-in-programming

Encapsulation: Core Principles

- **Data Hiding**: Protecting the internal state of an object.
- **Controlled Access**: Providing methods (getters and setters) to access and modify data.
- **Modularity**: Breaking down a system into independent, manageable units.
- **Abstraction**: Hiding complex implementation details and presenting a simplified interface.

Benefits of Encapsulation

- **Data Integrity**: Prevents accidental modification of data.
- **Enhanced Security**: Reduces the risk of unauthorized access.
- **Modularity**: Makes code easier to understand, test, and maintain.
- **Flexibility**: Allows internal implementation to change without affecting external code.
- **Reduced** Complexity: Simplifies the overall system by hiding unnecessary details.



Encapsulation in Action: Rectangle Class

```python
class Rectangle:
    def __init__(self, width, height):
        self.__width = width  # Private attribute
        self.__height = height # Private attribute

    def get_width(self):
        return self.__width

    def set_width(self, width):
        if width > 0:
            self.__width = width
        else:
            print("Width must be positive.")

    def get_height(self):
        return self.__height

    def set_height(self, height):
        if height > 0:
            self.__height = height
        else:
            print("Height must be positive.")

    def calculate_area(self):
        return self.__width * self.__height

    def display_dimensions(self):
        print(f"Width: {self.__width}, Height: {self.__height}")

rect = Rectangle(5, 10)
print(rect.get_width())  # Output: 5
rect.set_width(7)
print(rect.calculate_area())  # Output: 70
rect.display_dimensions()  # Output: Width: 7, Height: 10
```

Think-Pair-Share: Why Getters and Setters?

- **Question**: Why do we use getters and setters instead of directly accessing the width and height attributes in the Rectangle class?

* Take 2 minutes to think about it individually, then pair up with a classmate and discuss your answers for 3 minutes.

```python
class Rectangle:
    def __init__(self, width, height):
        self.__width = width  # Private attribute
        self.__height = height # Private attribute

    def get_width(self):
        return self.__width

    def set_width(self, width):
        if width > 0:
            self.__width = width
        else:
            print("Width must be positive.")

    def get_height(self):
        return self.__height

    def set_height(self, height):
        if height > 0:
            self.__height = height
        else:
            print("Height must be positive.")

    def calculate_area(self):
        return self.__width * self.__height

    def display_dimensions(self):
        print(f"Width: {self.__width}, Height: {self.__height}")

rect = Rectangle(5, 10)
print(rect.get_width())  # Output: 5
rect.set_width(7)
print(rect.calculate_area())  # Output: 70
rect.display_dimensions()  # Output: Width: 7, Height: 10
```

Getters and Setters: Key Benefits

- **Data Validation**: Setters can validate input to ensure data integrity.
- **Controlled Access**: Allows read-only access or restricted modification.
- **Future Flexibility**: Internal implementation can change without affecting external code using the class.
- **Encapsulation**: Protects the internal state of the object.

Access Modifiers in Python

- Python doesn't have strict access modifiers like **private**, **protected**, and **public** found in other languages (e.g., Java, C++). Instead, it uses **naming conventions** to indicate the intended level of access.

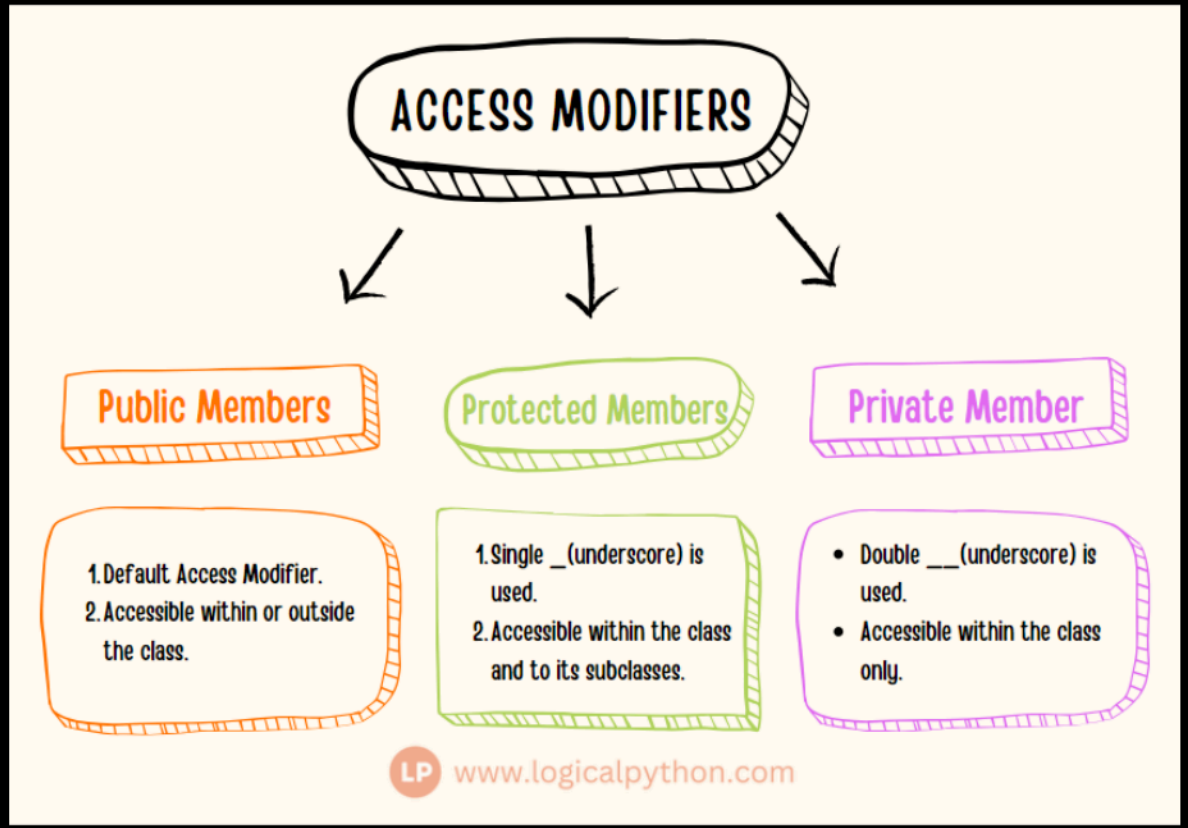


Image from Logicalpython, 2024, https://www.logicalpython.com/python-encapsulation-and-access-modifiers/

Public Access

- **Public** attributes and methods are accessible from anywhere – both inside and outside the class. No special naming convention is required.

```python
class Dog:
    def __init__(self, name):
        self.name = name  # Public attribute

    def bark(self): # Public method
        print("Woof!")


my_dog = Dog("Buddy")
print(my_dog.name) # Accessing public attribute
my_dog.bark() # Calling public method
```

Protected Access

- **Protected** attributes and methods are intended for internal use within the class and its subclasses. They are indicated by a **single underscore prefix _**.
- It's a signal to other developers that these should not be accessed directly from outside the class.

```python
class Vehicle:
    def __init__(self):
        self._engine_type = "Gasoline"  # Protected attribute

    def _start_engine(self): # Protected Method
        print("Engine Starting")

class Car(Vehicle):
    def display_engine_type(self):
        print(f"Engine type: {self._engine_type}")

my_car = Car()
my_car.display_engine_type()  # Accessing protected attribute from subclass
```

Private Access

- **Private** attributes and methods are intended for internal use only within the class. They are indicated by a **double underscore prefix** __.
- **Name Mangling**: Python uses name mangling to make these attributes harder (but not impossible) to access from outside the class.

```python
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # Private attribute

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(100)
# print(account.__balance)  # This will raise an AttributeError
print(account._BankAccount__balance) # Accessing using name mangling, but discouraged
```

Name Mangling: How it Works

- When Python encounters an attribute or method name with a double underscore prefix, it changes the name to *_ClassName__AttributeName*.
- Purpose: To avoid accidental name collisions in subclasses and to provide a degree of privacy.
- Example: In the *BankAccount* class, *__balance* becomes *_BankAccount__balance*.

```
1   class BankAccount:
2       def __init__(self, balance):
3           self.__balance = balance  # Private attribute
4
5       def deposit(self, amount):
6           self.__balance += amount
7
8       def get_balance(self):
9           return self.__balance
10
11  account = BankAccount(100)
12  # print(account.__balance)  # This will raise an AttributeError
13  print(account._BankAccount__balance) # Accessing using name mangling, but discouraged
```

Benefits of Name Mangling

- **Reduces Name Collisions**: Avoids conflicts when subclasses have attributes with the same name as superclasses.
- **Encourages Good Design**: Signals to developers that an attribute is intended for internal use.
- **Provides a Degree of Privacy**: Makes it harder (but not impossible) to access private attributes from outside the class.

When to Use Name Mangling

- **Preventing Name Collisions**: When you want to avoid accidental name collisions in subclasses.
- **Indicating Internal Use**: To signal to developers that an attribute or method is intended for internal use only.
- **Protecting Sensitive Data**: To make it harder for external code to access sensitive data.

```python
class Parent:
    def __init__(self):
        self.__private_attribute = "Parent's Private"

    def get_private(self):
        return self.__private_attribute

class Child(Parent):
    def __init__(self):
        super().__init__()
        self.__private_attribute = "Child's Private"

    def get_child_private(self):
         return self.__private_attribute

parent = Parent()
child = Child()

print(parent.get_private())  # Output: Parent's Private
print(child.get_private())   # Output: Parent's Private (from Parent class)
print(child.get_child_private()) # Output: Child's Private
# Accessing mangled names
print(child._Parent__private_attribute) # Output: Parent's Private
print(child._Child__private_attribute) # Output: Child's Private

```

Name Mangling: Cautions

- **Not True Privacy**: It doesn't provide absolute privacy; attributes can still be accessed using name mangling.
- **Overuse**: Overusing name mangling can make code harder to read and understand.
- **Testability**: Can make unit testing more difficult if you rely heavily on accessing private attributes.

- Python's Philosophy: **"We are all responsible users**"
  - Python relies on the principle that developers are responsible and will respect the intended access levels.
  - Even private attributes can be accessed using name mangling.
  - It's a matter of convention and good practice, not strict enforcement.

https://docs.python-guide.org/writing/style/#we-are-all-responsible-users

Information Hiding: Beyond Encapsulation

- **Information hiding** is the principle of **concealing** the internal implementation details of a module or class from the outside world.
- To reduce **complexity**, improve **maintainability**, and increase **flexibility**.
- Encapsulation is a mechanism to achieve information hiding.

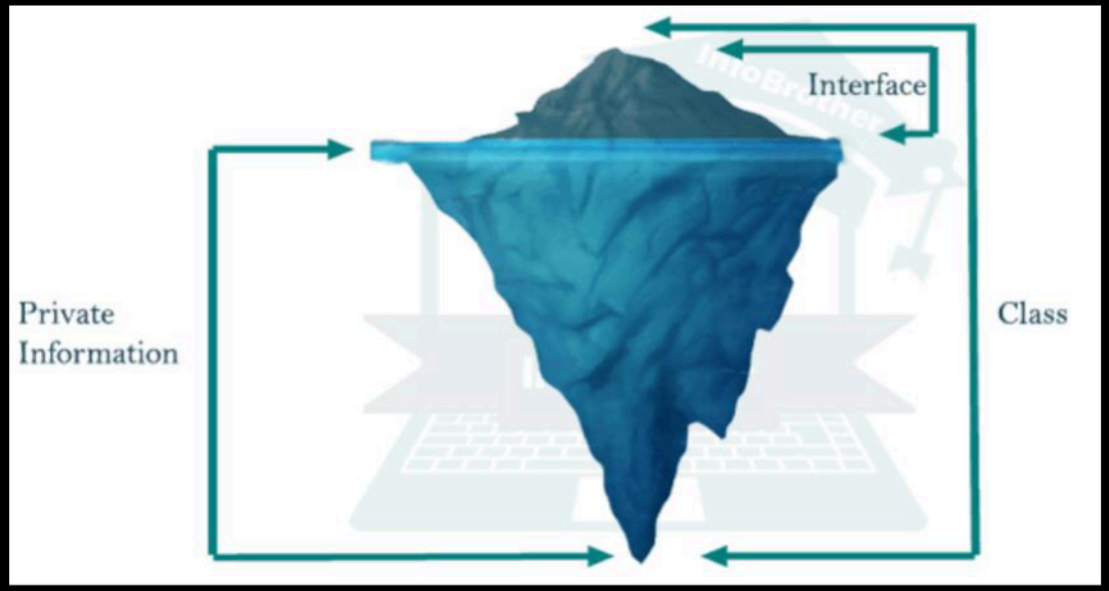


Image from Muntazir Fadhel, 2019, https://mfadhel.com/object-oriented-microservices-migrations/

Techniques for Information Hiding

- **Use Private Attributes**: Hide internal data using the __ prefix.
- **Implement Public Interfaces**: Provide well-defined methods for interacting with the class.
- **Abstract Complex Logic**: Hide complex calculations or algorithms behind simple method calls.
- **Use Helper Methods**: Break down complex methods into smaller, private helper methods.

Information Hiding Example: Temperature Converter

```python
class TemperatureConverter:
    def __init__(self):
        pass

    def celsius_to_fahrenheit(self, celsius):
        return (celsius * 9/5) + 32

    def fahrenheit_to_celsius(self, fahrenheit):
        return (fahrenheit - 32) * 5/9

    # Internal function that should be private
    def __kelvin_to_celsius(self, kelvin):
        return kelvin - 273.15

    def convert_to_standard(self, kelvin, standard="Celsius"):
        celsius = self.__kelvin_to_celsius(kelvin)
        if standard == "Fahrenheit":
            return self.celsius_to_fahrenheit(celsius)
        return celsius

converter = TemperatureConverter()
print(converter.celsius_to_fahrenheit(25))  # Output: 77.0
print(converter.fahrenheit_to_celsius(77))  # Output: 25.0
```

Benefits of Information Hiding

- **Reduced Complexity**: Simplifies the interface for users of the class.
- **Increased Maintainability**: Allows internal implementation to be changed without affecting external code.
- **Improved Security**: Prevents unauthorized access to sensitive data.
- **Enhanced Flexibility**: Makes it easier to adapt the class to new requirements.

Getters and Setters: Controlling Access

- **Getters** (also known as accessors) are methods used to retrieve the value of an attribute.
- **Setters** (also known as mutators) are methods used to modify the value of an attribute.
- Purpose: To provide controlled access to an object's attributes and to enforce data validation.

- OOP and encapsulation are not remedies for bad architecture! If all your classes need access to each others private variables, the problem is not OOP.

Getter and Setter Example

- The *get_name()* method provides read-only access to the *__name* attribute.
- The *set_name()* method validates the input before modifying the *__name* attribute.

```python
class Person:
    def __init__(self, name):
        self.__name = name

    def set_name(self, new_name):
        if isinstance(new_name, str) and len(new_name) > 0:
            self.__name = new_name
        else:
            print("Invalid name.")

    def get_name(self):
        return self.__name

person = Person("Alice")
person.set_name("Bob")
print(person.get_name())  # Output: Bob
```

Pythonic Getters and Setters: Properties

- Python provides a more elegant way to define getters and setters using the *@property* decorator.
- The *@property* decorator defines the getter, and the *@radius.setter* decorator defines the setter.
- Question: How to use *@property* to stop others from accessing the radius?

```python
class Circle:
    def __init__(self, radius):
        self.__radius = radius

    @property
    def radius(self):
        return self.__radius

    @radius.setter
    def radius(self, value):
        if value > 0:
            self.__radius = value
        else:
            print("Radius must be positive.")

circle = Circle(5)
print(circle.radius)  # Output: 5
circle.radius = 7
print(circle.radius)  # Output: 7
```

Think-Pair-Share: Why Properties?

- Question: What are the advantages of using properties compared to traditional getter and setter methods?

```python
class Circle:
    def __init__(self, radius):
        self.__radius = radius

    @property
    def radius(self):
        return self.__radius

    @radius.setter
    def radius(self, value):
        if value > 0:
            self.__radius = value
        else:
            print("Radius must be positive.")

circle = Circle(5)
print(circle.radius)  # Output: 5
circle.radius = 7
print(circle.radius)  # Output: 7
```

Properties: Advantages

- **Readability**: Makes code more concise and easier to understand.
- **Maintainability**: Simplifies the process of adding validation or other logic to attribute access.
- **Flexibility**: Allows you to change the implementation of attribute access without affecting external code.

Encapsulation Best Practices

- Use **Access Modifiers** (Conventions): Follow the public, protected, and private naming conventions.
- Use **Getters** and **Setters** (Properties): Provide controlled access to attributes.
- **Hide Implementation Details**: Conceal complex logic and data structures.
- Favor **Immutability**: When possible, make objects immutable to prevent accidental modification.
- **Document** Your Code: Clearly document the intended use of attributes and methods.

Immutability: A Powerful Tool

- Immutable objects **cannot** be modified after they are created.
- Benefits:
  - **Simplified Reasoning**: Easier to understand and reason about the behaviour of immutable objects.
  - **Thread Safety**: Immutable objects are inherently thread-safe.
  - **Reduced Errors**: Prevents accidental modification of data.

```python
from collections import namedtuple

Point = namedtuple("Point", ["x", "y"])
p1 = Point(1, 2)
# p1.x = 3  # This will raise an AttributeError because Point is immutable
p2 = Point(3,4)
```

The Importance of Documentation

- **Clear and concise documentation** is crucial for understanding and maintaining encapsulated code.
- What to Document:
  - Purpose of attributes and methods.
  - Intended access levels (public, protected, private).
  - Any validation rules or constraints.
  - Any potential side effects.

Refactoring for Better Encapsulation

- Scenario: You have existing code that doesn't follow encapsulation principles.
- Steps:
  - Identify attributes that should be **private**.
  - Create **getters** and **setters** (properties) for controlled access.
  - Move complex logic into **private helper methods**.
  - Add **documentation** to explain the intended use of attributes and methods.

Encapsulation: Common Pitfalls

- **Over-Encapsulation**: Making everything private can lead to unnecessary complexity.
- **Under-Encapsulation**: Exposing too much internal state can lead to data corruption and maintenance problems.
- **Ignoring Conventions**: Not following the public, protected, and private naming conventions.
- **Not Documenting Your Code**: Makes it difficult for others (and yourself!) to understand and maintain the code.

Encapsulation: Inside Your Car

- Designing a Car class to represent your vehicle.

- *model* and *color*: Public. These are external characteristics readily observable and known to anyone who sees the car. It's the overall appearance!
- *__engine_running* and *__fuel_level*: Private. These represent the internal state of the car's mechanics. You don't directly manipulate these from outside, but control them through methods like *start_engine()*, *stop_engine()*, and *refuel()*.

```python
class Car:
    def __init__(self, model, color):
        self.model = model  # Public: Everyone knows what model it is
        self.color = color  # Public: Everyone can see the colour
        self.__engine_running = False  # Private: Start the engine with the key
        self.__fuel_level = 100  # Private: Internal fuel level

    def start_engine(self, car_key):
        if car_key == "car_key_number":
            self.__engine_running = True
            print("Vroom! Engine started.")

    def stop_engine(self):
        self.__engine_running = False
        print("Silence. Engine stopped.")

    def drive(self, distance):
        if self.__engine_running:
            if self.__fuel_level > 0:
                print(f"Driving {distance} miles.")
                self.__fuel_level -= distance / 10  # Simulating fuel consumption
                if self.__fuel_level < 0:
                    self.__fuel_level = 0
                print(f"Fuel level: {self.__fuel_level}%")
            else:
                print("Out of fuel!  Need to refuel.")
        else:
            print("Engine is not running. Start the engine first.")

    def refuel(self, amount):
        self.__fuel_level += amount
        if self.__fuel_level > 100:
            self.__fuel_level = 100
        print(f"Refueled. Fuel level: {self.__fuel_level}%")


# Example Usage
my_car = Car("Tesla Model 3", "Red")
print(f"My car is a {my_car.color} {my_car.model}") #publicly shown
my_car.start_engine() #start engines
my_car.drive(50) #drive
my_car.stop_engine() #stop the engine
```

Key Takeaways

- Encapsulation is bundling and hiding data.
- Access modifiers in Python are convention-based (public, protected, private).
- Information hiding reduces complexity and improves maintainability.
- Getters and setters (properties) provide controlled access to attributes.
- Name mangling helps prevent name collisions and provides a degree of privacy.
- Following best practices is crucial for writing well-encapsulated code.

Q & A

VICTORIA UNIVERSITY
MELBOURNE AUSTRALIA

Lab